

Python Lab Questions - Complete Solutions

Part 1: General Questions

Q1. Write a program to print "Hello, World!".

```
print("Hello, World!")
```

Output / Sample Run:

Hello, World!

Q2. Take user input for a name and print a personalized greeting.

```
name = input("Enter your name: ")
```

```
print(f"Hello, {name}!")
```

Output / Sample Run:

Enter your name: Arom

Hello, Arom!

Q3. Calculate the sum of two numbers.

```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter second number: "))
```

```
print("Sum =", a + b)
```

Output / Sample Run:

Enter first number: 15

Enter second number: 25

Sum = 40

Q4. Calculate the product of three floating-point numbers.

```
a = float(input("Enter first number: "))
```

```
b = float(input("Enter second number: "))
```

```
c = float(input("Enter third number: "))
```

```
print("Product =", a * b * c)
```

Output / Sample Run:

Enter first number: 2.5

Enter second number: 2.0

Enter third number: 4.0

Product = 20.0

Q5. Calculate simple interest from principal, rate, and time.

```
p = float(input("Principal: "))
```

```
r = float(input("Rate: "))
```

```
t = float(input("Time: "))
```

```
si = (p * r * t) / 100
```

```
print("Simple Interest =", si)
```

Output / Sample Run:

Principal: 10000

Rate: 5

Time: 2

Simple Interest = 1000.0

Q6. Calculate area and perimeter of a rectangle.

```
l = float(input("Length: "))
```

```
w = float(input("Width: "))
```

```
print("Area =", l * w)
```

```
print("Perimeter =", 2 * (l + w))
```

```
Output / Sample Run:
Length: 10
Width: 5
Area = 50.0
Perimeter = 30.0
```

Q7. Calculate area and circumference of a circle.

```
import math
r = float(input("Radius: "))
print("Area =", math.pi * r * r)
print("Circumference =", 2 * math.pi * r)

Output / Sample Run:
Radius: 7
Area = 153.93804002589985
Circumference = 43.982297150257104
```

Q8. Calculate the volume of a cylinder.

```
import math
r = float(input("Radius: "))
h = float(input("Height: "))
volume = math.pi * r * r * h
print("Volume =", volume)

Output / Sample Run:
Radius: 3
Height: 5
Volume = 141.3716694115407
```

Q9. Calculate the average of three numbers.

```
a = float(input("First: "))
b = float(input("Second: "))
c = float(input("Third: "))
print("Average =", (a + b + c) / 3)

Output / Sample Run:
First: 10
Second: 20
Third: 30
Average = 20.0
```

Q10. Convert Celsius to Fahrenheit.

```
c = float(input("Celsius: "))
f = (9/5) * c + 32
print("Fahrenheit =", f)

Output / Sample Run:
Celsius: 25
Fahrenheit = 77.0
```

Note: The source text displays the fraction formatting imperfectly; the standard conversion is $F = (9/5)C + 32$.

Q11. Convert Fahrenheit to Celsius.

```
f = float(input("Fahrenheit: "))
c = (5/9) * (f - 32)
print("Celsius =", c)

Output / Sample Run:
Fahrenheit: 98.6
Celsius = 37.0
```

Q12. Convert kilometers to meters and centimeters.

```
km = float(input("Kilometers: "))
print("Meters =", km * 1000)
print("Centimeters =", km * 100000)
```

```
Output / Sample Run:
Kilometers: 2.5
Meters = 2500.0
Centimeters = 25000.0
```

Q13. Convert USD to EUR.

```
usd = float(input("USD: "))
rate = 0.92 # example rate: 1 USD = 0.92 EUR
print("EUR =", usd * rate)
```

```
Output / Sample Run:
USD: 100
EUR = 92.0
```

Note: The source does not give a USD/EUR exchange rate, so this uses an explicitly labeled example rate.

Q14. Convert Nepali Rupees to Indian Currency (1 IC = 1.6 NRs).

```
nrs = float(input("Nepali Rupees: "))
ic = nrs / 1.6
print("Indian Currency =", ic)
```

```
Output / Sample Run:
Nepali Rupees: 160
Indian Currency = 100.0
```

Q15. Calculate the square and cube of a number.

```
n = float(input("Number: "))
print("Square =", n ** 2)
print("Cube =", n ** 3)
```

```
Output / Sample Run:
Number: 4
Square = 16.0
Cube = 64.0
```

Q16. Calculate remainder and quotient when dividing two integers.

```
a = int(input("Dividend: "))
b = int(input("Divisor: "))
print("Quotient =", a // b)
print("Remainder =", a % b)
```

```
Output / Sample Run:
Dividend: 17
Divisor: 5
Quotient = 3
Remainder = 2
```

Q17. Swap two variables using a third variable.

```
a = 10
b = 20
temp = a
a = b
b = temp
print("a =", a)
print("b =", b)
```

```
Output / Sample Run:
a = 20
b = 10
```

Q18. Swap two variables without using a third variable.

```
a = 10
b = 20
a, b = b, a
print("a =", a)
print("b =", b)

Output / Sample Run:
a = 20
b = 10
```

Q19. Calculate total seconds from hours, minutes, and seconds.

```
h = int(input("Hours: "))
m = int(input("Minutes: "))
s = int(input("Seconds: "))
total = h*3600 + m*60 + s
print("Total seconds =", total)

Output / Sample Run:
Hours: 2
Minutes: 30
Seconds: 15
Total seconds = 9015
```

Q20. Convert total seconds into hours, minutes, and remaining seconds.

```
total = int(input("Total seconds: "))
h = total // 3600
rem = total % 3600
m = rem // 60
s = rem % 60
print("Hours =", h, "Minutes =", m, "Seconds =", s)

Output / Sample Run:
Total seconds: 9015
Hours = 2 Minutes = 30 Seconds = 15
```

Part 2: Conditional Statements

Q21. Check whether an integer is even or odd.

```
n = int(input("Enter integer: "))
print("Even" if n % 2 == 0 else "Odd")

Output / Sample Run:
Enter integer: 8
Even
```

Q22. Check whether a number is positive, negative, or zero.

```
n = float(input("Enter number: "))
if n > 0:
    print("Positive")
elif n < 0:
    print("Negative")
else:
    print("Zero")

Output / Sample Run:
Enter number: -4
Negative
```

Q23. Find the maximum between two numbers.

```
a = float(input("a: "))
b = float(input("b: "))
print("Maximum =", max(a, b))

Output / Sample Run:
a: 12
b: 9
Maximum = 12.0
```

Q24. Find the minimum between two numbers.

```
a = float(input("a: "))
b = float(input("b: "))
print("Minimum =", min(a, b))

Output / Sample Run:
a: 12
b: 9
Minimum = 9.0
```

Q25. Find the largest of three numbers.

```
a, b, c = map(float, input("Enter 3 numbers: ").split())
print("Largest =", max(a, b, c))

Output / Sample Run:
Enter 3 numbers: 12 45 30
Largest = 45.0
```

Q26. Find the smallest of three numbers.

```
a, b, c = map(float, input("Enter 3 numbers: ").split())
print("Smallest =", min(a, b, c))

Output / Sample Run:
Enter 3 numbers: 12 45 30
Smallest = 12.0
```

Q27. Check voting eligibility (age >= 18).

```
age = int(input("Age: "))
print("Eligible to vote" if age >= 18 else "Not eligible to vote")

Output / Sample Run:
Age: 20
Eligible to vote
```

Q28. Check whether a year is a leap year.

```
year = int(input("Year: "))
if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):
    print("Leap year")
else:
    print("Not a leap year")

Output / Sample Run:
Year: 2024
Leap year
```

Q29. Check whether a character is a vowel or consonant.

```
ch = input("Character: ").lower()
if ch.isalpha():
    print("Vowel" if ch in "aeiou" else "Consonant")
else:
    print("Not an alphabet")

Output / Sample Run:
Character: e
Vowel
```

Q30. Check whether a character is an alphabet, digit, or special character.

```
ch = input("Character: ")
if ch.isalpha():
    print("Alphabet")
elif ch.isdigit():
    print("Digit")
else:
    print("Special character")
```

Output / Sample Run:
Character: @
Special character

Q31. Check whether an alphabet is uppercase or lowercase.

```
ch = input("Alphabet: ")
if ch.isalpha():
    print("Uppercase" if ch.isupper() else "Lowercase")
else:
    print("Not an alphabet")
```

Output / Sample Run:
Alphabet: G
Uppercase

Q32. Check whether a number is divisible by both 5 and 11.

```
n = int(input("Number: "))
print("Divisible by both 5 and 11" if n % 5 == 0 and n % 11 == 0 else "Not divisible by both")
```

Output / Sample Run:
Number: 55
Divisible by both 5 and 11

Q33. Check whether a number is divisible by 3 or 7, but not both.

```
n = int(input("Number: "))
result = (n % 3 == 0) ^ (n % 7 == 0)
print("Condition satisfied" if result else "Condition not satisfied")
```

Output / Sample Run:
Number: 21
Condition not satisfied

Q34. Check whether three angles form a valid triangle.

```
a, b, c = map(float, input("Enter 3 angles: ").split())
print("Valid triangle" if a > 0 and b > 0 and c > 0 and a+b+c == 180 else "Invalid triangle")
```

Output / Sample Run:
Enter 3 angles: 60 60 60
Valid triangle

Q35. Check whether three side lengths form a valid triangle.

```
a, b, c = map(float, input("Enter 3 sides: ").split())
print("Valid triangle" if a+b>c and a+c>b and b+c>a else "Invalid triangle")
```

Output / Sample Run:
Enter 3 sides: 3 4 5
Valid triangle

Q36. Classify a triangle as equilateral, isosceles, or scalene.

```
a, b, c = map(float, input("Enter 3 sides: ").split())
if a == b == c:
    print("Equilateral")
elif a == b or b == c or a == c:
    print("Isosceles")
else:
    print("Scalene")
```

Output / Sample Run:
Enter 3 sides: 5 5 3
Isosceles

Q37. Calculate quadratic roots and identify real/equal/complex roots.

```
import cmath
a = float(input("a: ")); b = float(input("b: ")); c = float(input("c: "))
d = b*b - 4*a*c
r1 = (-b + cmath.sqrt(d)) / (2*a)
r2 = (-b - cmath.sqrt(d)) / (2*a)
print("Roots:", r1, r2)
if d > 0: print("Real and distinct")
elif d == 0: print("Real and equal")
else: print("Complex")
```

Output / Sample Run:
a: 1
b: -5
c: 6
Roots: (3+0j) (2+0j)
Real and distinct

Q38. Calculate a student's grade based on percentage.

```
p = float(input("Percentage: "))
if p >= 90: grade = "A"
elif p >= 80: grade = "B"
elif p >= 70: grade = "C"
elif p >= 60: grade = "D"
else: grade = "F"
print("Grade =", grade)
```

Output / Sample Run:
Percentage: 86
Grade = B

Note: The source only gives A>=90, B>=80 and says "etc."; C/D/F cutoffs shown here are an explicit example.

Q39. Calculate electricity bill using tiered tariff rates.

```
units = float(input("Units used: "))
if units <= 20:
    bill = units * 4
elif units <= 50:
    bill = 20*4 + (units-20)*6
elif units <= 100:
    bill = 20*4 + 30*6 + (units-50)*8
else:
    bill = 20*4 + 30*6 + 50*8 + (units-100)*10
print("Bill = Rs.", bill)
```

Output / Sample Run:
Units used: 80
Bill = Rs. 440.0

Note: The source does not provide tariff values, so these rates are an explicit example only.

Q40. Calculate gross salary using basic salary, HRA, and DA tiers.

```
basic = float(input("Basic salary: "))
if basic <= 30000:
    hra_rate, da_rate = 0.20, 0.10
elif basic <= 60000:
    hra_rate, da_rate = 0.15, 0.12
else:
    hra_rate, da_rate = 0.10, 0.15
hra = basic * hra_rate
da = basic * da_rate
print("HRA =", hra)
print("DA =", da)
print("Gross salary =", basic + hra + da)
```

Output / Sample Run:
Basic salary: 50000
HRA = 7500.0
DA = 6000.0
Gross salary = 63500.0

Note: The source does not specify the HRA/DA tiers, so the shown percentages are an explicit example.

Q41. Input a month number and display the number of days.

```
month = int(input("Month (1-12): "))
if month == 2:
    days = 28 # ignoring leap years
elif month in [4, 6, 9, 11]:
    days = 30
elif 1 <= month <= 12:
    days = 31
else:
    days = None
print("Days =", days if days else "Invalid month")
```

Output / Sample Run:
Month (1-12): 4
Days = 30

Q42. Input a day number (1-7) and display the day of week.

```
days = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"]
n = int(input("Day number (1-7): "))
print(days[n-1] if 1 <= n <= 7 else "Invalid day number")
```

Output / Sample Run:
Day number (1-7): 2
Monday

Q43. Create a basic console menu for add, subtract, and exit.

```
while True:
    print("1. Add\n2. Subtract\n3. Exit")
    choice = int(input("Choice: "))
    if choice == 3:
        print("Exiting..."); break
    a = float(input("a: ")); b = float(input("b: "))
    if choice == 1: print("Result =", a+b)
    elif choice == 2: print("Result =", a-b)
    else: print("Invalid choice")
```

Output / Sample Run:

```
1. Add
2. Subtract
3. Exit
Choice: 1
a: 15
b: 6
Result = 21.0
Choice: 3
Exiting...
```

Q44. Calculate net amount after percentage discount rules based on total purchase amount.

```
amount = float(input("Purchase amount: "))
if amount >= 10000: rate = 20
elif amount >= 5000: rate = 10
elif amount >= 2000: rate = 5
else: rate = 0
discount = amount * rate / 100
print("Discount =", discount)
print("Net amount =", amount - discount)
```

Output / Sample Run:

```
Purchase amount: 6000
Discount = 600.0
Net amount = 5400.0
```

Note: The source does not specify discount brackets; these are explicit example rules.

Q45. Check whether a 3-digit number starts and ends with the same digit.

```
n = input("Enter a 3-digit number: ")
if len(n) == 3 and n.isdigit():
    print("Same" if n[0] == n[-1] else "Different")
else:
    print("Invalid 3-digit number")
```

Output / Sample Run:

```
Enter a 3-digit number: 121
Same
```

Part 3: Looping & Iteration

Q46. Print numbers from 1 to 10 using a loop.

```
for i in range(1, 11):
    print(i, end=" ")
```

Output / Sample Run:

```
1 2 3 4 5 6 7 8 9 10
```

Q47. Print numbers from 10 down to 1.

```
for i in range(10, 0, -1):
    print(i, end=" ")
```

Output / Sample Run:

```
10 9 8 7 6 5 4 3 2 1
```

Q48. Print all even numbers between 1 and 50.

```
for i in range(2, 51, 2):
    print(i, end=" ")
```

Output / Sample Run:

```
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50
```

Q49. Print all odd numbers between 1 and 50.

```
for i in range(1, 51, 2):
    print(i, end=" ")

Output / Sample Run:
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49
```

Q50. Calculate the sum of first N natural numbers.

```
n = int(input("N: "))
total = 0
for i in range(1, n+1): total += i
print("Sum =", total)

Output / Sample Run:
N: 10
Sum = 55
```

Q51. Calculate the sum of all even numbers between 1 and N.

```
n = int(input("N: "))
total = 0
for i in range(2, n+1, 2): total += i
print("Sum =", total)

Output / Sample Run:
N: 10
Sum = 30
```

Q52. Calculate factorial of a number.

```
n = int(input("N: "))
f = 1
for i in range(1, n+1): f *= i
print("Factorial =", f)

Output / Sample Run:
N: 5
Factorial = 120
```

Q53. Display multiplication table up to 10.

```
n = int(input("Number: "))
for i in range(1, 11): print(f"{n} x {i} = {n*i}")

Output / Sample Run:
Number: 5
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50
```

Q54. Display all factors of a number.

```
n = int(input("Number: "))
for i in range(1, n+1):
    if n % i == 0: print(i, end=" ")

Output / Sample Run:
Number: 12
1 2 3 4 6 12
```

Q55. Count the total number of factors of a number.

```
n = int(input("Number: "))
count = 0
for i in range(1, n+1):
    if n % i == 0: count += 1
print("Number of factors =", count)
```

Output / Sample Run:

Number: 12

Number of factors = 6

Q56. Check whether a number is prime or composite.

```
n = int(input("Number: "))
if n < 2: print("Neither prime nor composite")
else:
    prime = True
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: prime = False; break
    print("Prime" if prime else "Composite")
```

Output / Sample Run:

Number: 29

Prime

Q57. Print all prime numbers between 1 and 100.

```
for n in range(2, 101):
    prime = True
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: prime = False; break
    if prime: print(n, end=" ")
```

Output / Sample Run:

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

Q58. Find GCD/HCF of two integers.

```
import math
a, b = map(int, input("Enter two integers: ").split())
print("GCD =", math.gcd(a, b))
```

Output / Sample Run:

Enter two integers: 24 36

GCD = 12

Q59. Find LCM of two integers.

```
import math
a, b = map(int, input("Enter two integers: ").split())
lcm = abs(a*b) // math.gcd(a, b) if a and b else 0
print("LCM =", lcm)
```

Output / Sample Run:

Enter two integers: 12 18

LCM = 36

Q60. Count the total number of digits in an integer.

```
n = int(input("Number: "))
print("Digits =", len(str(abs(n))))
```

Output / Sample Run:

Number: -12345

Digits = 5

Q61. Calculate sum of all digits.

```
n = abs(int(input("Number: ")))
total = 0
while n:
    total += n % 10; n //= 10
print("Digit sum =", total)
```

Output / Sample Run:

Number: 12345

Digit sum = 15

Q62. Calculate product of all digits.

```
n = abs(int(input("Number: ")))
product = 1
if n == 0: product = 0
while n:
    product *= n % 10; n //= 10
print("Digit product =", product)

Output / Sample Run:
Number: 1234
Digit product = 24
```

Q63. Calculate sum of only even digits.

```
n = abs(int(input("Number: ")))
total = sum(int(d) for d in str(n) if int(d) % 2 == 0)
print("Sum of even digits =", total)

Output / Sample Run:
Number: 123456
Sum of even digits = 12
```

Q64. Calculate product of only odd digits.

```
n = abs(int(input("Number: ")))
product = 1
found = False
for d in str(n):
    if int(d) % 2 == 1: product *= int(d); found = True
print("Product of odd digits =", product if found else 0)

Output / Sample Run:
Number: 1356
Product of odd digits = 15
```

Q65. Reverse a given integer.

```
n = int(input("Number: "))
s = str(n)
sign = "-" if s.startswith("-") else ""
print("Reversed =", int(sign + s.lstrip("-")[::-1]))

Output / Sample Run:
Number: 1234
Reversed = 4321
```

Q66. Check whether an integer is a palindrome.

```
n = input("Number: ")
print("Palindrome" if n == n[::-1] else "Not palindrome")

Output / Sample Run:
Number: 1221
Palindrome
```

Q67. Check whether an integer is an Armstrong number.

```
n = input("Number: ")
p = len(n)
s = sum(int(d)**p for d in n)
print("Armstrong" if s == int(n) else "Not Armstrong")

Output / Sample Run:
Number: 153
Armstrong
```

Q68. Check whether a number is a Perfect Number.

```
n = int(input("Number: "))
s = sum(i for i in range(1, n) if n % i == 0)
print("Perfect number" if s == n else "Not a perfect number")

Output / Sample Run:
Number: 28
Perfect number
```

Q69. Generate AP series 1,4,7,10,13,... up to N terms.

```
n = int(input("Number of terms: "))
term = 1
for _ in range(n):
    print(term, end=" "); term += 3

Output / Sample Run:
Number of terms: 6
1 4 7 10 13 16
```

Q70. Generate square series up to the 10th term.

```
for i in range(1, 11):
    print(i*i, end=" ")

Output / Sample Run:
1 4 9 16 25 36 49 64 81 100
```

Q71. Generate geometric series 1,5,25,125,... up to 7 terms.

```
term = 1
for _ in range(7):
    print(term, end=" "); term *= 5

Output / Sample Run:
1 5 25 125 625 3125 15625
```

Q72. Generate 1,2,4,7,11,16,... up to 10 terms.

```
term = 1
diff = 1
for _ in range(10):
    print(term, end=" "); term += diff; diff += 1

Output / Sample Run:
1 2 4 7 11 16 22 29 37 46
```

Q73. Generate 1,2,5,10,17,26,... up to 10 terms.

```
term = 1
diff = 1
for _ in range(10):
    print(term, end=" "); term += diff; diff += 2

Output / Sample Run:
1 2 5 10 17 26 37 50 65 82
```

Q74. Generate Fibonacci series up to N terms.

```
n = int(input("Number of terms: "))
a, b = 0, 1
for _ in range(n):
    print(a, end=" "); a, b = b, a+b

Output / Sample Run:
Number of terms: 8
0 1 1 2 3 5 8 13
```

Q75. Generate pattern series 1,121,12321,1234321,... up to 10 terms.

```
n = 10
for i in range(1, n+1):
    left = "".join(str(j) for j in range(1, i+1))
    print(left + left[-2::-1] if i > 1 else left)
```

Output / Sample Run:

```
1
121
12321
1234321
123454321
12345654321
1234567654321
123456787654321
12345678987654321
12345678910987654321
```

Part 4: Functions & Modular Programming

Q76. Define a function that accepts two numbers and returns their sum.

```
def add(a, b):
    return a + b
print(add(10, 20))
```

Output / Sample Run:

```
30
```

Q77. Function returning True if a number is even, else False.

```
def is_even(n):
    return n % 2 == 0
print(is_even(8))
```

Output / Sample Run:

```
True
```

Q78. Function to convert Celsius to Fahrenheit.

```
def c_to_f(c):
    return (9/5) * c + 32
print(c_to_f(25))
```

Output / Sample Run:

```
77.0
```

Q79. Function returning factorial.

```
def factorial(n):
    result = 1
    for i in range(1, n+1): result *= i
    return result
print(factorial(5))
```

Output / Sample Run:

```
120
```

Q80. Function returning sum of first N natural numbers.

```
def natural_sum(n):
    return sum(range(1, n+1))
print(natural_sum(10))
```

Output / Sample Run:

```
55
```

Q81. Function checking whether a number is prime.

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True
print(is_prime(29))
Output / Sample Run:
True
```

Q82. Function returning GCD of two integers.

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return abs(a)
print(gcd(24, 36))
Output / Sample Run:
12
```

Q83. Function returning largest among three numbers.

```
def largest(a, b, c):
    return max(a, b, c)
print(largest(10, 25, 15))
Output / Sample Run:
25
```

Q84. Function returning the longest of three strings.

```
def longest(a, b, c):
    return max(a, b, c, key=len)
print(longest("cat", "computer", "school"))
Output / Sample Run:
computer
```

Q85. Function returning the number of digits in an integer.

```
def digit_count(n):
    return len(str(abs(n)))
print(digit_count(12345))
Output / Sample Run:
5
```

Q86. Function returning the reversed value of an integer.

```
def reverse_num(n):
    s = str(n)
    return int(("-" if s.startswith("-") else "") + s.lstrip("-")[::-1])
print(reverse_num(1234))
Output / Sample Run:
4321
```

Q87. Function checking Armstrong number using modular sub-functions.

```
def digit_power_sum(n):
    s = str(n); p = len(s)
    return sum(int(d)**p for d in s)
def is_armstrong(n):
    return n >= 0 and digit_power_sum(n) == n
print(is_armstrong(153))
Output / Sample Run:
True
```

Q88. Function returning total number of vowels in a string.

```
def count_vowels(text):
    return sum(1 for ch in text.lower() if ch in "aeiou")
print(count_vowels("Hello Nepal"))

Output / Sample Run:
4
```

Q89. Function returning total number of consonants in a string.

```
def count_consonants(text):
    return sum(1 for ch in text.lower() if ch.isalpha() and ch not in "aeiou")
print(count_consonants("Hello Nepal"))

Output / Sample Run:
6
```

Q90. Function that prints all non-vowel characters.

```
def print_consonants(text):
    for ch in text:
        if ch.isalpha() and ch.lower() not in "aeiou": print(ch, end=" ")
print_consonants("Nepal")

Output / Sample Run:
N p l
```

Q91. Count how many times the first character repeats throughout the string.

```
def first_char_count(text):
    return text.count(text[0]) if text else 0
print(first_char_count("banana"))

Output / Sample Run:
3
```

Q92. Transform a string into alternating uppercase/lowercase characters.

```
def alternate_case(text):
    return "".join(ch.upper() if i % 2 == 0 else ch.lower() for i, ch in enumerate(text))
print(alternate_case("nepal"))

Output / Sample Run:
NePaL
```

Q93. Check whether a string is a palindrome.

```
def is_palindrome(text):
    return text == text[::-1]
print(is_palindrome("madam"))

Output / Sample Run:
True
```

Q94. Calculate x^y using iteration without a power operator.

```
def power(x, y):
    result = 1
    for _ in range(y): result *= x
    return result
print(power(2, 5))

Output / Sample Run:
32
```

Q95. Recursive function to compute factorial.

```
def factorial(n):
    if n <= 1: return 1
    return n * factorial(n-1)
print(factorial(5))
```

Output / Sample Run:
120

Q96. Recursive function to return the N-th Fibonacci number.

```
def fib(n):  
    if n <= 1: return n  
    return fib(n-1) + fib(n-2)  
print(fib(7))
```

Output / Sample Run:
13

Q97. Recursive function to compute sum of digits.

```
def digit_sum(n):  
    n = abs(n)  
    if n < 10: return n  
    return n % 10 + digit_sum(n // 10)  
print(digit_sum(12345))
```

Output / Sample Run:
15

Q98. Modular program with functions for area and perimeter based on menu choice.

```
import math  
def rectangle(l, w): return l*w, 2*(l+w)  
def circle(r): return math.pi*r*r, 2*math.pi*r  
print("1. Rectangle\n2. Circle")  
choice = int(input("Choice: "))  
if choice == 1:  
    l, w = map(float, input("Length Width: ").split()); print("Area, Perimeter =", rectangle(l,w))  
elif choice == 2:  
    r = float(input("Radius: ")); print("Area, Circumference =", circle(r))
```

Output / Sample Run:

1. Rectangle

2. Circle

Choice: 1

Length Width: 10 5

Area, Perimeter = (50.0, 30.0)

Q99. Function accepting a list and returning its maximum element.

```
def list_max(values):  
    return max(values)  
print(list_max([10, 25, 7, 19]))
```

Output / Sample Run:
25

Q100. Function returning only prime numbers from a list.

```
def is_prime(n):  
    if n < 2: return False  
    return all(n % i for i in range(2, int(n**0.5)+1))  
def prime_list(values):  
    return [n for n in values if is_prime(n)]  
print(prime_list([2, 4, 5, 9, 11, 15]))
```

Output / Sample Run:
[2, 5, 11]

Part 5: Standard Libraries (math, random)

Q101. Use math to find square root of 49.

```
import math
print(math.sqrt(49))
```

Output / Sample Run:
7.0

Q102. Find the square root of a user-entered number using math.

```
import math
n = float(input("Number: "))
print("Square root =", math.sqrt(n))
```

Output / Sample Run:
Number: 81
Square root = 9.0

Q103. Import math and calculate pi multiplied by 10.

```
import math
print(math.pi * 10)
```

Output / Sample Run:
31.41592653589793

Q104. Use random to print a random integer between 1 and 10.

```
import random
print(random.randint(1, 10))
```

Output / Sample Run:
Example output: 7

Note: The exact value changes on each run.

Q105. Generate a random number between 1 and 10.

```
import random
number = random.randint(1, 10)
print("Random number =", number)
```

Output / Sample Run:
Example output: Random number = 4

Note: The exact value changes on each run.

Q106. Roll a die 5 times using random and a for loop.

```
import random
for _ in range(5):
    print(random.randint(1, 6))
```

Output / Sample Run:
Example output:
4
1
6
2
5

Note: The exact sequence changes on each run.

Q107. Study the given random.randint(1, 5) code and state its output.

```
import random
print(random.randint(1, 5))
```

Output / Sample Run:
An integer from 1 through 5, for example: 3

Note: Because it is random, one exact output cannot be guaranteed.

Part 6: Graphics Using Turtle

Q108. Draw a filled circle of radius 50.

```
import turtle
t = turtle.Turtle()
t.begin_fill()
t.circle(50)
t.end_fill()
turtle.done()
```

Output / Sample Run:

A filled circle of radius 50 is displayed in a Turtle graphics window.

Q109. Draw a 100-unit line, turn left 90 degrees, and draw a 50-unit line.

```
import turtle
t = turtle.Turtle()
t.forward(100)
t.left(90)
t.forward(50)
turtle.done()
```

Output / Sample Run:

Two perpendicular line segments are displayed; the second segment is 50 units after a 90-degree left turn.

Q110. Draw a square using Turtle.

```
import turtle
t = turtle.Turtle()
for _ in range(4):
    t.forward(100)
    t.right(90)
turtle.done()
```

Output / Sample Run:

A square is displayed.

Q111. Draw a colored square with blue pen and red fill.

```
import turtle
t = turtle.Turtle()
t.color("blue")
t.fillcolor("red")
t.begin_fill()
for _ in range(4):
    t.forward(100)
    t.right(90)
t.end_fill()
turtle.done()
```

Output / Sample Run:

A square with a blue outline and red interior is displayed.

Q112. Draw a hexagon using a for loop and calculate the turn angle dynamically.

```
import turtle
t = turtle.Turtle()
sides = 6
angle = 360 / sides
for _ in range(sides):
    t.forward(80)
    t.right(angle)
turtle.done()
```

Output / Sample Run:

A regular hexagon is displayed. The turn angle is 60 degrees.

Q113. Analyze the given 3-side Turtle code.

```
for i in range(3):
    turtle.forward(100)
    turtle.left(120)
```

Output / Sample Run:
It draws an equilateral triangle.

Q114. Point out the logic error in the Turtle square code.

```
import turtle
t = turtle.Turtle()
for i in range(4):
    t.forward(80)
    t.right(90)
# Correct turn angle: 90 degrees
```

Output / Sample Run:
The error is `t.right(80)`. A square needs a 90-degree turn after each side.

Q115. Study the Turtle program with `t.right(90)` and answer the two questions.

```
import turtle
t = turtle.Turtle()
for i in range(4):
    t.forward(100)
    t.right(90)
turtle.done()
```

Output / Sample Run:
(a) Shape: Square.
(b) `right(90)` turns the turtle 90 degrees clockwise.

Part 7: Errors and Exception Handling

Q116. Catch `ValueError` when a user enters non-numeric text for an integer input.

```
try:
    n = int(input("Enter an integer: "))
    print("You entered", n)
except ValueError:
    print("Invalid integer input")
```

Output / Sample Run:
Enter an integer: abc
Invalid integer input

Q117. Handle division by zero using `try-except`.

```
try:
    a = int(input("a: "))
    b = int(input("b: "))
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output / Sample Run:
a: 10
b: 0
Cannot divide by zero

Q118. Take two integers, divide them, and catch ZeroDivisionError with the specified message.

```
try:
    a = int(input("First: "))
    b = int(input("Second: "))
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero!")

Output / Sample Run:
First: 25
Second: 0
Cannot divide by zero!
```

Q119. Handle FileNotFoundError when reading a missing file.

```
try:
    with open("missing.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found")

Output / Sample Run:
File not found
```

Q120. State the error from marks = [45,67,80]; print(marks[3]).

```
marks = [45, 67, 80]
print(marks[3])

Output / Sample Run:
IndexError: list index out of range
```

Q121. Explain the purpose of the try block in the supplied division program.

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")

Output / Sample Run:
(a) The try block contains statements that might raise an exception.
(b) ZeroDivisionError occurs when the second number is 0 and division is attempted.
```

Part 8: File Handling, Pandas, and CSV Files

Q122. State the result of opening data.txt in read mode and then calling file.write("Hello").

```
file = open("data.txt", "r")
file.write("Hello")

Output / Sample Run:
io.UnsupportedOperation: not writable
```

Note: The exact first exception if data.txt does not exist would instead be FileNotFoundError. Assuming the file exists, write() in read mode raises io.UnsupportedOperation.

Q123. Open records.txt in append mode, write New Data, and close it.

```
file = open("records.txt", "a")
file.write("New Data")
file.close()

Output / Sample Run:
No console output. The text "New Data" is appended to records.txt.
```

Q124. Load inventory.csv into df using one Pandas command.

```
import pandas as pd
df = pd.read_csv("inventory.csv")
```

Output / Sample Run:
No console output; df contains the CSV data.

Q125. Create data.txt and write the test sentence.

```
with open("data.txt", "w") as file:
    file.write("Hello, this is a test file.")
```

Output / Sample Run:
No console output. data.txt now contains: Hello, this is a test file.

Q126. Append the sentence to data.txt.

```
with open("data.txt", "a") as file:
    file.write(" This is an appended line.")
```

Output / Sample Run:
No console output. The sentence is appended to the file.

Q127. Read data.txt and print its contents.

```
with open("data.txt", "r") as file:
    print(file.read())
```

Output / Sample Run:
Hello, this is a test file. This is an appended line.

Q128. Create students.csv with Name and Marks headers and three records using csv module.

```
import csv
with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["Name", "Marks"])
    writer.writerows([["Asha", 85], ["Bikash", 72], ["Chandra", 91]])
```

Output / Sample Run:
No console output. students.csv is created with 3 student records.

Q129. Read students.csv with Pandas and display the first 5 rows.

```
import pandas as pd
df = pd.read_csv("students.csv")
print(df.head())
```

Output / Sample Run:

	Name	Marks
0	Asha	85
1	Bikash	72
2	Chandra	91

Q130. Create a DataFrame with Name and Marks and save to marks.csv.

```
import pandas as pd
df = pd.DataFrame({"Name": ["Asha", "Bikash", "Chandra"], "Marks": [85, 72, 91]})
df.to_csv("marks.csv", index=False)
```

Output / Sample Run:
No console output. marks.csv is created.

Q131. Read marks.csv and print records where Computer > 80.

```
import pandas as pd
df = pd.read_csv("marks.csv")
print(df[df["Computer"] > 80])
```

Output / Sample Run:

Example output:

	Name	Math	Computer
0	Asha	88	92
2	Chandra	95	85

Q132. Answer the file-handling snippet using open("test.txt", "w").

```
file = open("test.txt", "w")
file.write("Hello World")
file.close()
```

Output / Sample Run:

(a) Mode: w (write mode).

(b) write() writes the given string into the file; in w mode it creates/truncates the file first.

Q133. Predict the supplied Pandas filtering output.

```
import pandas as pd
data = {"Name": ["Asha", "Bikash", "Chandra"], "Marks": [45, 72, 60]}
df = pd.DataFrame(data)
print(df[df["Marks"] > 60])
```

Output / Sample Run:

	Name	Marks
1	Bikash	72

Data Visualization (Matplotlib & Plotly)

Q134. Generate a simple line plot with x=[1,2,3] and y=[10,20,30].

```
import matplotlib.pyplot as plt
x = [1, 2, 3]
y = [10, 20, 30]
plt.plot(x, y)
plt.show()
```

Output / Sample Run:

A line graph is displayed through the points (1,10), (2,20), and (3,30).

Q135. Plot daily temperatures over 5 days as a line chart.

```
import matplotlib.pyplot as plt
days = ["Sun", "Mon", "Tue", "Wed", "Thu"]
temps = [24, 26, 25, 27, 28]
plt.plot(days, temps, marker="o")
plt.xlabel("Day")
plt.ylabel("Temperature (C)")
plt.title("Daily Temperature")
plt.show()
```

Output / Sample Run:

A line chart showing the five daily temperatures is displayed.

Q136. Draw a bar chart of student marks across 4 subjects.

```
import matplotlib.pyplot as plt
subjects = ["Math", "Science", "Computer", "English"]
marks = [85, 78, 92, 88]
plt.bar(subjects, marks)
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.title("Student Marks")
plt.show()
```

Output / Sample Run:

A bar chart with four subject bars is displayed.

Q137. Read product sales from CSV with Pandas and show a pie chart.

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv("sales.csv")
plt.pie(df["Sales"], labels=df["Product"], autopct="%1.1f%%")
plt.title("Product Sales")
plt.show()
```

Output / Sample Run:

A pie chart showing each product's share of total sales is displayed.

Q138. Plot products A,B,C with sales 150,300,250 as a labeled bar chart.

```
import matplotlib.pyplot as plt
products = ["A", "B", "C"]
sales = [150, 300, 250]
plt.bar(products, sales)
plt.xlabel("Products")
plt.ylabel("Sales")
plt.title("Product Sales")
plt.show()
```

Output / Sample Run:

A bar chart is displayed with A=150, B=300, and C=250.

Q139. Predict the line-plot output and modify it to display a bar graph instead.

```
import matplotlib.pyplot as plt
months = ["Jan", "Feb", "Mar"]
sales = [50, 70, 60]
plt.bar(months, sales)
plt.show()
```

Output / Sample Run:

Original code: a line graph for Jan=50, Feb=70, Mar=60.

Modified code: replace `plt.plot(months, sales)` with `plt.bar(months, sales)` to display a bar graph.